# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE NAME: Database Management Systems**
**COURSE CODE: CSA05**

---

**COURSE OUTCOME COVERED**

**CO1:** *Analyze DBMS architecture, different data models, schema types, and design ER/EER diagrams, relational schemas for case-based scenarios by identifying entities and relationships.* (BL1, BL2)

Activity Tool : Short Answer Text(High)
Assessment Tool Weightage: 40%

| QUESTIONS | | |
|---|---|---|
| **Q. No** | **Question** | **Bloom's Level** |
| 1 | Explain schema and instance with examples. Distinguish between logical schema, physical schema, and view schema. | BL1 – Remember |
| 2 | Identify the entities, attributes, and relationships for a Bank Management System and construct its ER diagram. | BL2 – Understand |
| 3 | Explain the Extended ER (EER) model. Describe the concepts of generalization, specialization, and aggregation. | BL2 – Understand |
| 4 | Design an EER diagram for a Hospital Management System incorporating inheritance and weak entities. | BL2 – Understand |
| 5 | Explain the role of a DBA (Database Administrator) and the responsibilities associated with the position. | BL1 – Remember |

---

***Code of Conduct:***

I CH. Durga Prasad (192425305) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

***Signature of the student:***

**RUBRICS FOR CALCULATION:**

| **Criteria** | **Weightage** | **Level 4 (Exemplary)** | **Level 3 (Proficient)** | **Level 2 (Developing)** | **Level 1 (Beginning)** |
|---|---|---|---|---|---|
| Concept Identification | 30% | Demonstrates clear and complete understanding of the concept | Good understanding with minor gaps | Basic understanding with some misconceptions | Poor or incorrect understanding |
| Linkages | 25% | Completely correct answer with no errors | Mostly correct with minor errors | Partially correct with noticeable errors | Incorrect or irrelevant answer |
| Organization | 20% | Correctly applies concepts to solve the question/problem | Applies concepts with minor mistakes | Limited or partially correct application | No or incorrect application |
| Integration of Literature | 15% | Covers all required parts of the question | Covers most parts with minor omissions | Covers only some parts | Incomplete or missing response |
| Clarity | 10% | Clear, concise, and well-organized answer | Understandable with minor issues | Somewhat unclear or poorly structured | Difficult to understand |

## 1. Explain schema and instance with examples. Distinguish between logical schema, physical schema, and view schema.

**Introduction:**

Before storing data in a database, its structure must be defined. DBMS uses the concepts of schema and instance to distinguish between the database design and the actual stored data. Understanding different types of schemas helps in designing secure and efficient database systems.

**Schema:**

A database schema is the logical blueprint or overall structure of a database. It defines how data is organized, including tables, attributes, relationships, constraints, and data types. A schema is designed during database creation and changes very rarely. An instance is the actual data stored in the database at a particular moment. Unlike the schema, the instance changes frequently whenever records are inserted, updated, or deleted.

**Example:**

**Schema: Student Table**

| Student_ID | Name | Department | CGPA |
|---|---|---|---|

The above table structure represents the schema.

**Instance:**

| Student_ID | Name | Department | CGPA |
|---|---|---|---|
| 101 | Rahul | CSE | 8.9 |
| 102 | Priya | IT | 9.2 |

The above records represent the **instance**.

## Difference between Schema and Instance:

| Schema | Instance |
|---|---|
| Blueprint of the database | Actual data stored in the database |
| Changes rarely | Changes frequently |
| Created during database design | Changes during database operations |
| Defines structure | Contains data values |

## Types of Schema:

**1. Logical Schema**

- Describes the logical structure of the database.
- Defines tables, attributes, primary keys, foreign keys, and relationships.
- Independent of physical storage.

Example: Student(Student_ID, Name, Department, CGPA)

**2. Physical Schema**

- Describes how data is physically stored in storage devices.
- Includes indexing, file organization, hashing, and storage allocation.
- Managed by the DBMS.

**3. View Schema (External Schema)**

- Provides customized views of the database for different users.
- Improves data security by restricting access.

Example: A faculty member can view only Student_ID, Name, and CGPA, while the administrator can view all details.

**Conclusion**

Schema defines the structure of the database, whereas an instance represents the current data. The logical, physical, and view schemas together form the three-schema architecture of a DBMS.

## 2. Identify the entities, attributes, and relationships for a Bank Management System and construct its ER diagram.

**Introduction**

A Bank Management System (BMS) is a database application that manages customer information, bank accounts, loans, and branch details efficiently. It helps banks perform daily operations such as opening accounts, processing loans, maintaining customer records, and managing branch activities. To design such a database, the Entity-Relationship (ER) model is used. An ER diagram visually represents the entities, attributes, and relationships within the system, making the database structure easier to understand and implement.

**Entities and Attributes**

An entity is a real-world object or concept that can be uniquely identified. Each entity has a set of attributes that describe it.

**1. Customer**

Represents a bank customer.

Attributes:

- Customer_ID (Primary Key)
- Name
- Address
- Phone_Number
- Email

**2. Account**

Represents the bank account owned by customers.

Attributes:

- Account_Number (Primary Key*)*
- Account_Type
- Balance
- Opening_Date

**3. Branch**

Represents a bank branch.

Attributes:

- Branch_ID (Primary Key)
- Branch_Name
- Location

**4. Loan**

Represents the loans provided by the bank.

Attributes:

- Loan_ID (Primary Key)
- Loan_Type
- Loan_Amount
- Interest_Rate

**Relationships**

The relationships between the entities are:

**Customer – Owns – Account (1:M)**

- One customer can own one or more accounts.

- Each account belongs to one customer.

**Account – Belongs To – Branch (M:1)**

- Many accounts are maintained in one branch.
- Each account belongs to only one branch.

**Customer – Applies – Loan (1:M)**

- One customer can apply for multiple loans.
- Each loan is associated with one customer.

**ER Diagram:**

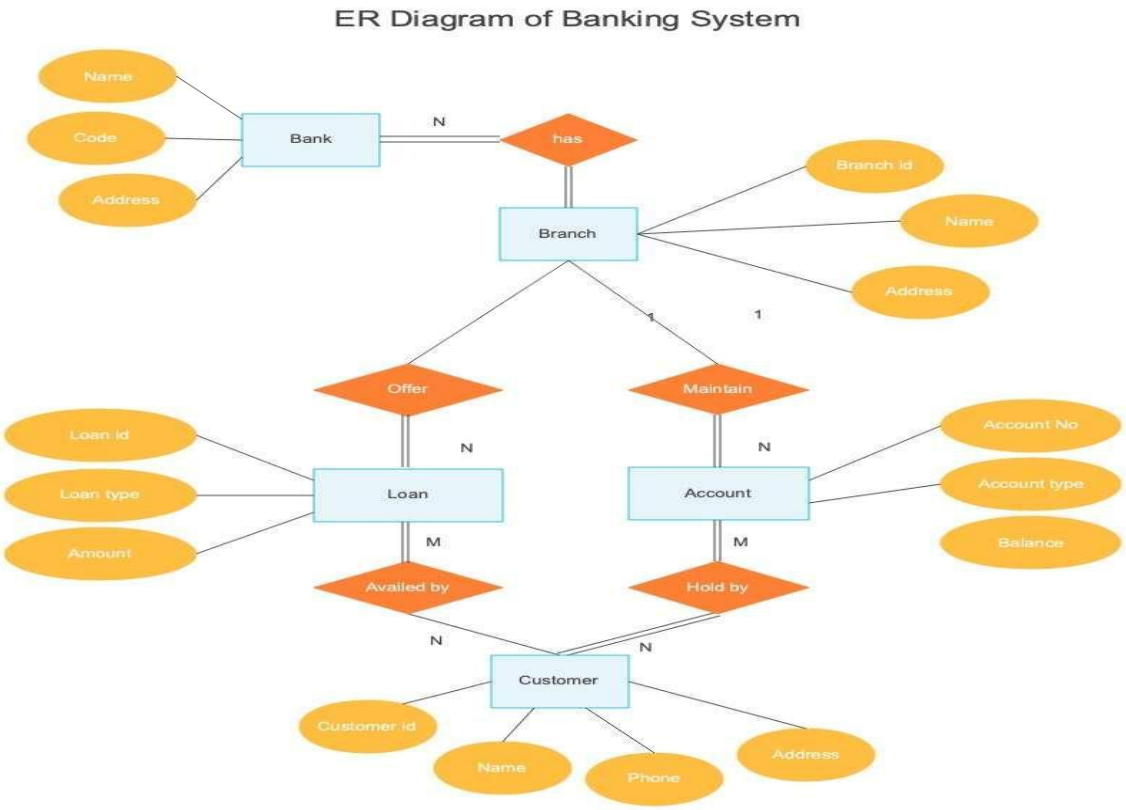


Fig.1:ER Diagram of Banking System

**Explanation of the ER Diagram:**

The ER diagram consists of four main entities:

**Customer**

The **Customer** entity stores personal information such as Customer_ID, Name, Address, Phone_Number, and Email. It is the central entity because customers interact with almost every banking service.

**Account**

The **Account** entity stores account details including Account_Number, Account_Type, Balance, and Opening_Date. A customer may own multiple accounts such as savings and current accounts.

**Branch**

The **Branch** entity contains details about each bank branch, including Branch_ID, Branch_Name, and Location. Every account is maintained by a specific branch.

**Loan**

The **Loan** entity stores information about loans such as Loan_ID, Loan_Type, Loan_Amount, and Interest_Rate. Customers can apply for one or more loans based on their financial requirements.

**Relationships**

- **Owns (1:M):** One customer can own multiple accounts, but each account belongs to only one customer.
- **Belongs To (M:1):** Multiple accounts are maintained by one branch, while each account belongs to only one branch.
- **Applies (1:M):** A customer can apply for multiple loans, but each loan belongs to one customer.

## Summary Table:

| Entity | Primary Key | Important Attributes |
|---|---|---|
| Customer | Customer_ID | Name, Address, Phone_Number, Email |
| Account | Account_Number | Account_Type, Balance, Opening_Date |
| Branch | Branch_ID | Branch_Name, Location |
| Loan | Loan_ID | Loan_Type, Loan_Amount, Interest_Rate |

## Conclusion:

The ER model of the Bank Management System clearly represents the database structure by identifying the main entities, their attributes, and the relationships among them. It provides a systematic representation of customer accounts, bank branches, and loans, ensuring efficient data storage, retrieval, and management. This design minimizes redundancy, maintains data integrity, and serves as the foundation for implementing a reliable banking database system.

## 3. Explain the Extended ER (EER) model. Describe the concepts of generalization, specialization, and aggregation.

## Introduction:

The Entity-Relationship (ER) model is widely used to design databases by representing entities, attributes, and relationships. However, for complex real-world applications such as hospitals, universities, banking systems, and e-commerce platforms, the traditional ER model is not sufficient. To overcome these limitations, the Extended Entity-Relationship (EER) model was introduced. The EER model extends the ER model by providing advanced features such as generalization, specialization, inheritance, aggregation, and categories, making database design more flexible and realistic.

## Extended Entity-Relationship (EER) Model

The Extended Entity-Relationship (EER) model is an enhanced version of the ER model that supports advanced modeling concepts. It allows designers to represent complex relationships, inheritance hierarchies, and higher-level abstractions found in real-world systems.

Features of the EER Model

- Supports inheritance (ISA relationship).
- Represents generalization and specialization.
- Supports aggregation for modeling complex relationships.
- Reduces data redundancy.
- Improves database organization and flexibility.
- Represents real-world objects more accurately than the traditional ER model.

## 1. Generalization:

**Definition**

**Generalization** is the process of combining two or more lower-level entities that share common attributes into a single higher-level entity called the **superclass**.

It follows a **bottom-up approach**, where common characteristics are identified and merged into one generalized entity.

**Example**

Consider two account types:

- Savings Account
- Current Account

Both share common attributes such as Account Number, Balance, and Opening Date. Therefore, they can be generalized into a single entity called **Account**.

**ER Diagram:**

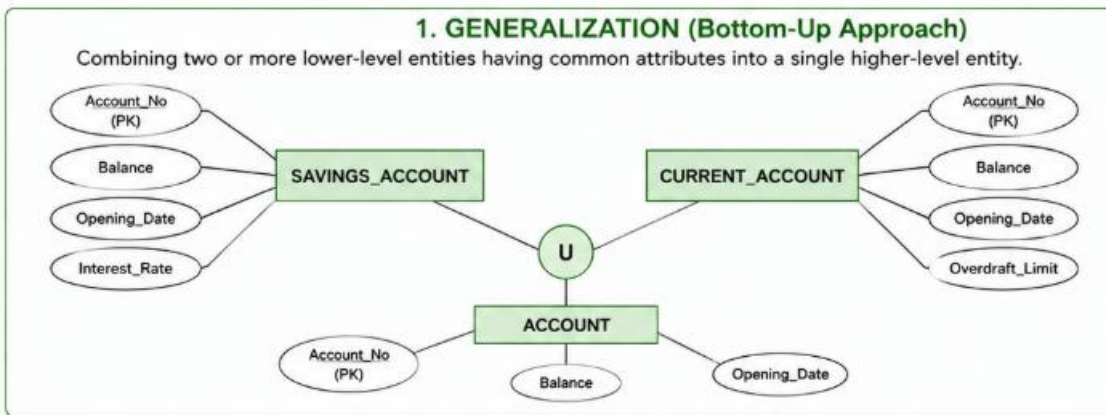


Fig.2: Generalization

## 2. Specialization:

**Definition**

**Specialization** is the process of dividing a higher-level entity (superclass) into multiple lower-level entities (subclasses) based on specific characteristics.

It follows a **top-down approach**, where a general entity is divided into more specific entities.

**Example**

An Employee entity can be specialized into:

- Doctor
- Nurse
- Receptionist

Each subclass inherits the common attributes of Employee and also contains its own unique attributes.

**Diagram**

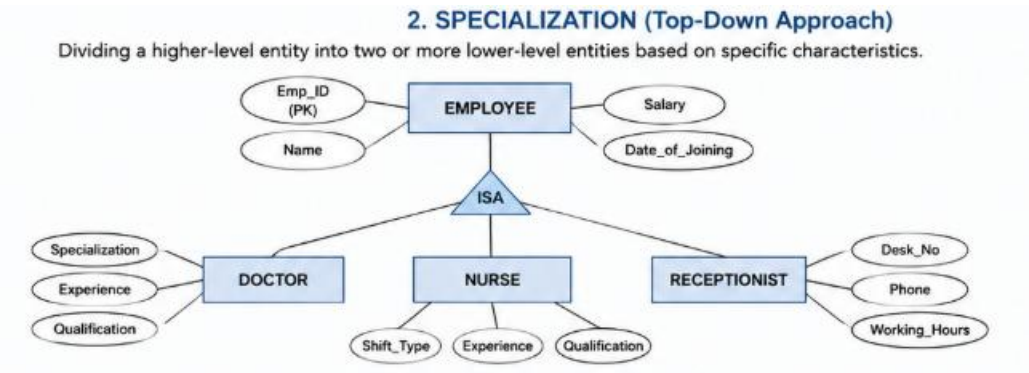


**Fig.3:Specialization**

**Advantages of Specialization**

- Represents different categories of an entity.
- Reduces redundancy.
- Supports inheritance.
- Makes database design more organized.

## 3. Aggregation

**Definition**

**Aggregation** is a technique in the EER model where a **relationship set is treated as a higher-level entity** so that it can participate in another relationship.

Aggregation is used when a relationship itself has to be related to another entity.

**Example**

An employee works on a project, and the department manages that work.

Instead of connecting the Department directly to Employee or Project, the relationship **Works_On** is aggregated.

**Diagram**

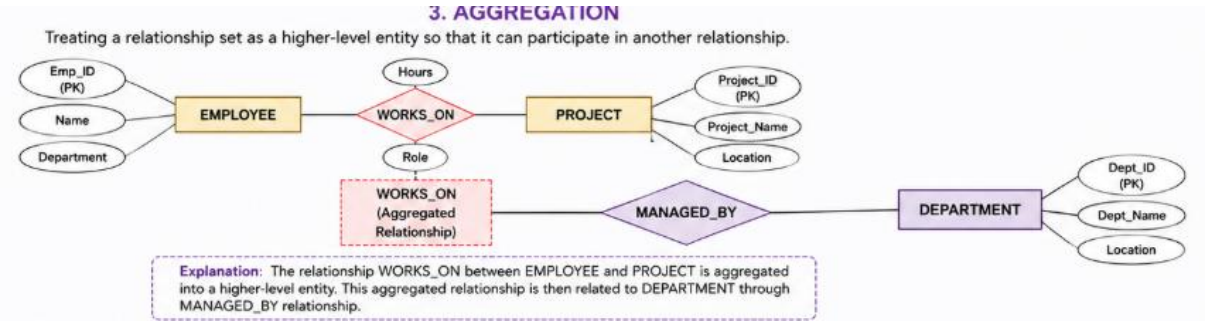


**Fig.4:Aggregation**

**Advantages of Aggregation**

- Represents complex relationships.
- Improves clarity in database design.
- Allows relationships to participate in other relationships.
- Simplifies complex database structures.

## Applications of the EER Model

The EER model is commonly used in:

- Hospital Management Systems
- Banking Systems
- University Management Systems
- Library Management Systems
- Airline Reservation Systems
- E-commerce Applications

## Conclusion

The Extended Entity-Relationship (EER) model is a powerful extension of the traditional ER model that provides advanced concepts for designing complex databases. Generalization combines similar entities into a superclass, specialization divides a superclass into specialized subclasses, and aggregation treats a relationship as a higher-level entity. These concepts reduce redundancy, improve data organization, support inheritance, and make database systems more efficient and easier to maintain.

# 4. Design an EER diagram for a Hospital Management System incorporating inheritance and weak entities.

## Introduction:

A Hospital Management System (HMS) is used to manage information related to patients, doctors, appointments, treatments, and medical records. Since hospitals contain complex data with shared and specialized information, the Extended Entity-Relationship (EER) model is more suitable than the traditional ER model. The EER model supports advanced concepts such as inheritance (ISA relationship) and weak entities, allowing the database to represent real-world hospital operations efficiently and accurately.

**Entities and Attributes**

## 1. Person (Super Class)

The Person entity stores common information shared by both doctors and patients.

Attributes

- Person_ID (Primary Key)
- Name
- Age
- Gender
- Address

## 2. Doctor (Subclass)

Doctor inherits all common attributes from Person and includes doctor-specific information.

Attributes

- Doctor_ID (Primary Key)
- Specialization
- Experience

## 3. Patient (Subclass)

Patient also inherits the common attributes from Person and stores patient-specific information.

Attributes

- Patient_ID (Primary Key)
- Disease
- Admission_Date

## 4. Appointment

Stores details of appointments between doctors and patients.

Attributes

- Appointment_ID (Primary Key)
- Appointment_Date
- Appointment_Time

## 5. Medical_Record (Weak Entity)

Medical_Record cannot exist without a Patient. It depends on the Patient entity for identification.

Attributes

- Record_No (Partial Key)
- Diagnosis
- Prescription

## Relationships

Inheritance (ISA Relationship)

- Person is the superclass.
- Doctor and Patient are subclasses.
- Both inherit common attributes from Person.

Doctor – Treats – Patient

- One doctor can treat many patients.
- One patient may be treated by multiple doctors.

Appointment

- Appointment records the interaction between Doctor and Patient.

Patient – Identifies – Medical_Record

- Each Patient can have multiple Medical Records.
- Medical_Record is a weak entity because it cannot exist independently.

## EER Diagram:

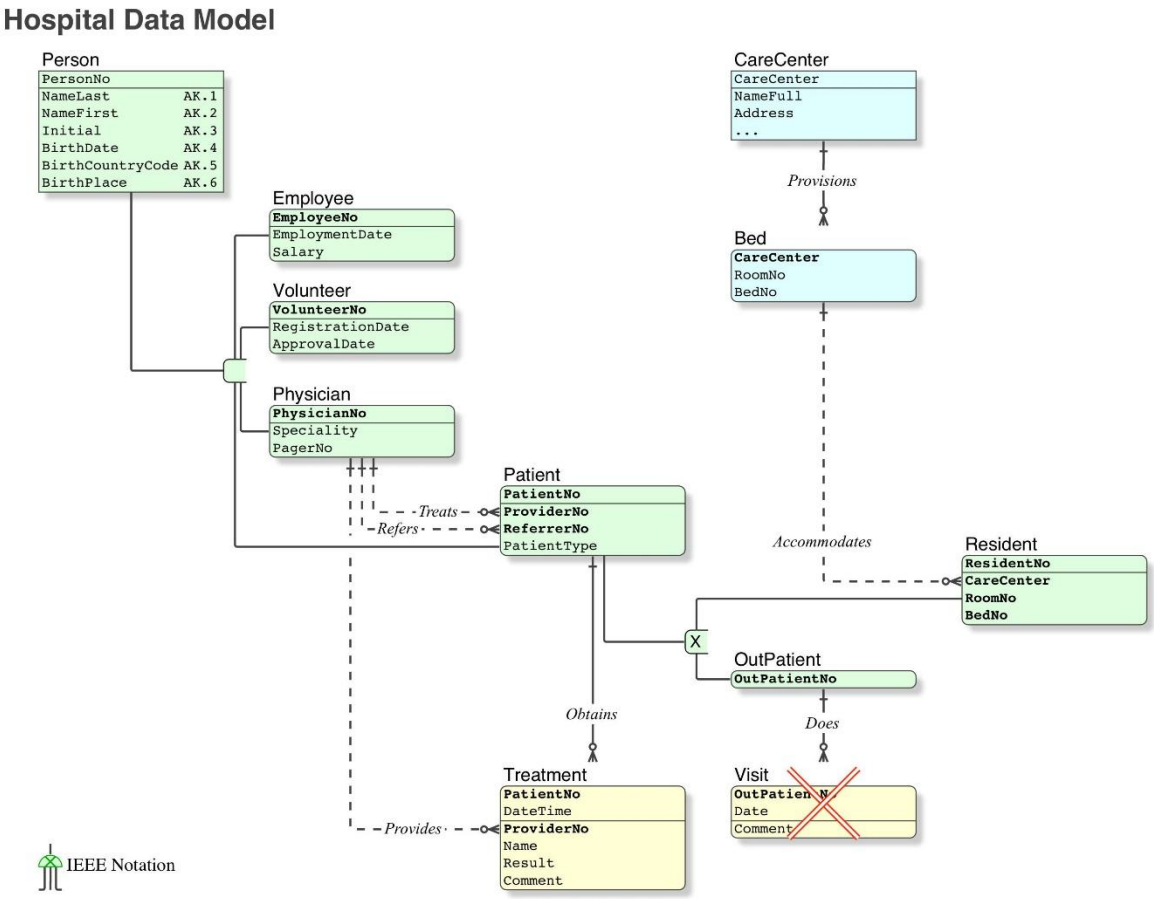


**Fig.5: Hospital Data**

## Explanation of the EER Diagram

### Person (Superclass)

The Person entity contains common attributes such as Person_ID, Name, Age, Gender, and Address. These attributes are shared by both doctors and patients.

### Inheritance (ISA)

The ISA (Is-A) relationship represents inheritance.

- Doctor inherits the common attributes of Person and adds Specialization and Experience.
- Patient inherits the common attributes of Person and adds Disease and Admission_Date.

This inheritance reduces data redundancy because common information is stored only once in the Person entity.

### Doctor–Patient Relationship

The Treats relationship connects doctors and patients.

- A doctor can treat many patients.

- A patient may receive treatment from one or more doctors.

**Appointment Entity**

The Appointment entity records the consultation details between a doctor and a patient, including the appointment date and time.

**Medical_Record (Weak Entity)**

The Medical_Record entity stores patient diagnosis and prescriptions.

It is called a weak entity because:

- It does not have a complete primary key of its own.
- It depends on the Patient entity for identification.
- It is uniquely identified using Patient_ID + Record_No (Partial Key).

## Conclusion

The Hospital Management System EER diagram effectively models real-world hospital operations by using inheritance and weak entities**.** The Person superclass eliminates redundancy by storing common attributes only once, while Doctor and Patient inherit these attributes through the ISA relationship. The Medical_Record is modeled as a weak entity because it depends on the Patient entity for its existence. This EER design improves data organization, maintains data integrity, minimizes redundancy, and provides a scalable structure for implementing a reliable hospital database system.

## 5. Explain the role of a DBA (Database Administrator) and the responsibilities associated with the position.

## Introduction:

A Database Management System (DBMS) is essential for storing, organizing, and managing large amounts of data. To ensure that a database operates efficiently, securely, and reliably, organizations appoint a Database Administrator (DBA). The DBA is responsible for managing the database throughout its lifecycle, including installation, maintenance, security, performance optimization, backup, and recovery. A skilled DBA ensures that organizational data is always available, accurate, and protected from unauthorized access or system failures.

**Role of a Database Administrator (DBA)**

A Database Administrator (DBA) is a professional responsible for the overall management and administration of databases within an organization. The DBA ensures that the database system

functions efficiently, securely, and reliably while supporting the organization's operational and business requirements.

The primary role of a DBA is to maintain database availability, protect sensitive information, optimize database performance, and ensure data integrity.

**Responsibilities of a DBA**

**1. Database Installation and Configuration**

- Installs and configures Database Management System (DBMS) software such as MySQL, Oracle, SQL Server, or PostgreSQL.
- Creates and initializes databases according to organizational requirements.
- Configures database parameters for optimal performance.

**2. Database Design and Implementation**

- Designs database schemas, tables, relationships, indexes, and constraints.
- Ensures efficient database structure based on user requirements.
- Implements normalization techniques to reduce data redundancy.

**3. Security Management**

- Creates user accounts and authentication mechanisms.
- Assigns user roles and access privileges.
- Prevents unauthorized access by implementing security policies.
- Encrypts sensitive data whenever necessary.

**4. Backup and Recovery**

- Performs regular database backups to prevent data loss.
- Develops disaster recovery plans.
- Restores databases quickly after hardware failures, software crashes, or accidental deletion.

**5. Performance Monitoring and Optimization**

- Monitors database performance continuously.
- Optimizes SQL queries for faster execution.
- Creates indexes to improve query performance.
- Identifies and resolves performance bottlenecks.

**6. Data Integrity and Consistency**

- Ensures that data remains accurate, valid, and consistent.

- Implements primary keys, foreign keys, and integrity constraints.
- Prevents duplicate and inconsistent data.

**7. Database Maintenance**

- Updates and upgrades DBMS software.
- Removes obsolete or unnecessary data.
- Monitors storage usage and manages database space.
- Performs routine maintenance to ensure smooth database operation.

**8. Disaster Recovery**

- Prepares strategies for handling unexpected failures.
- Ensures minimum downtime during system failures.
- Tests backup and recovery procedures periodically.

**9. User Support and Troubleshooting**

- Assists users in accessing databases.
- Resolves database-related issues and errors.
- Provides technical support and training to database users.

**Importance of a DBA**

A Database Administrator is important because they:

- Ensure database security and protect confidential information.
- Maintain high database availability and reliability.
- Improve database performance and efficiency.
- Prevent data loss through regular backups.
- Maintain data consistency and integrity.
- Support business operations by ensuring uninterrupted access to data.

## Conclusion

A Database Administrator (DBA) plays a vital role in managing and maintaining an organization's database systems. The DBA is responsible for database installation, security, backup and recovery, performance tuning, maintenance, and ensuring data integrity. By performing these responsibilities, the DBA ensures that databases remain secure, efficient, reliable, and always available to support organizational operations. Therefore, the DBA is a key contributor to the successful management of data in any modern information system.